

Auditing Keep-or-Revert Decisions in Self-Improving Agents

Anonymous Authors

Abstract—An AI coding agent can test several program versions before choosing one to submit. A final score does not tell a recipient which tests supported that choice, and matching file hashes does not show that the recorded decision follows from those tests. We present a decision record and an offline verifier that connect code versions, paired test results, statistical calculations, and keep-or-revert decisions. The verifier recalculates recorded statistics, checks the declared decision rules, and checks whether the submitted code matches a recorded version. We characterize these checks using twelve synthetic evidence packages: nine deliberately introduced faults, two valid controls, and one internally consistent rewrite of an unsigned final reward. Format checks reject one fault; adding file-hash checks rejects three; full verification rejects all nine. Both valid controls and the reward rewrite pass. The results identify checks that add value beyond file binding, while demonstrating that internal consistency is not evidence of authentic execution. A retained real-run record illustrates the practical evidence gap, but does not provide an additional end-to-end verifier evaluation.

Index Terms—decision provenance, agent accountability, offline verification

I. INTRODUCTION

Suppose an AI coding agent is asked to improve a program that plays 2048. It edits the program, plays test games, compares scores, and either keeps the new version or returns to an earlier one. It repeats this process and finally submits a program. RSI-Exam provides this kind of executable research setting: agents improve task programs using visible evaluations, and their submissions are graded on unseen data [1]. Here, self-improving refers to this iterative improvement of a task program, not modification of the language model’s weights.

Now suppose another researcher receives the submitted program and its final score. To check how it was selected, the researcher needs more than the score: Which earlier version was the comparison against? Which results belonged to each version? Did the recorded improvement justify keeping the candidate under the stated rule? Is the submitted program the version that was selected?

Ordinary logs may contain answers, but a recipient still has to check that they agree. File hashes help associate records with supplied files; they do not check the meaning of those records. For example, a record and its referenced files can all have matching hashes even though the recorded score difference is arithmetically wrong. This paper concerns such *decision consistency*: whether supplied measurements and the declared selection rule support the recorded keep-or-revert decision. It does not recover the agent’s private reasoning

or prove that the measurements genuinely came from an execution.

Our question is: *what can a recipient check about a recorded version decision beyond file format and hash agreement?* We contribute an executable decision record and offline checking procedure, then use controlled changes to evidence packages to identify which additional inconsistencies it detects. The contribution is a task-specific implementation and characterization, not a new general theory of provenance or a method for improving agent scores.

A. A real record shows the evidence gap

A retained record from an actual RSI run makes the problem concrete. A candidate reports a higher visible mean score than its parent, 48115 versus 38974.5, but is marked reverted without a recorded evaluation under the selection rule. The submitted version instead branches from that parent. The higher reported score alone cannot explain this choice, nor does it prove that reverting was wrong.

We can verify the retained experiment log’s hash and recover the declared branch history. But original code snapshots and per-seed measurements are absent from this retained export, so we cannot check their bytes or recalculate the decision’s statistical support. The companion’s Case R inspection reports these limits explicitly. This real-run example motivates what must be captured; the controlled experiment below tests the verifier on a different, fully supplied synthetic package.

II. RECORDING AND CHECKING A DECISION

A. What the producer hands over

Our evidence package is a directory containing saved code versions, test-result files, a decision log, the final submitted code, and a machine-readable summary record. We call this summary a *capsule*. The producing workflow records decisions during a run and constructs the capsule from the retained material afterward. A recipient runs the verifier on this directory without access to the agent’s runtime and without rerunning the game-playing program.

Each decision identifies five things:

- 1) The *parent* version being compared against and the new *candidate*.
- 2) The parent and candidate result files, labeled by role and bound by file hashes.
- 3) The score direction (higher or lower is better), statistic, confidence-interval procedure, random seed, and threshold.

- 4) The recorded calculation and decision: keep, revert, or await a confirmation evaluation.
- 5) A later confirmation decision, if one is required to resolve that pending state.

The capsule also links the submitted program to a recorded version. A version occurrence is distinct from its code contents: reverting can create another occurrence of the same code. Code identity follows the task’s staging rules, excluding Python bytecode caches; a separate whole-directory hash checks the supplied tree. These conventions avoid confusing a cache change with a program change.

B. A worked decision, from scores to submission

Table I follows the synthetic package used in our controlled experiment. v1 is the parent. Candidate v2 performs worse and is reverted. Candidate v3 is also derived from v1: its initial result is inconclusive, but a subsequent confirmation supports keeping it. The submitted program is v3.

TABLE I
RUNNING EXAMPLE FROM THE SYNTHETIC PACKAGE, NOT A REAL-RUN RESULT.
EACH ROW COMPARES THE CANDIDATE WITH v1.

Candidate/test	Mean delta	90% interval	Decision
v2 / initial	−318.75	[−382.5, −258.75]	Revert
v3 / initial	260	[−30, 583.75]	Pending
v3 / confirm	518.75	[467.5, 575]	Keep

The calculations use paired games: parent and candidate are evaluated on the same seed within each comparison. For a higher-is-better score, the mean improvement is

$$\hat{\Delta} = \frac{1}{n} \sum_{i=1}^n (s_{\text{candidate}}(i) - s_{\text{parent}}(i)).$$

The interval expresses uncertainty around the estimated improvement. The verifier recomputes the estimate and percentile-bootstrap interval from the supplied per-seed results and declared algorithm parameters. The example uses eight paired scores, 5000 resamples, and bootstrap seed 20260902. For a lower-is-better score the differences change sign.

In this package’s declared policy, an interval entirely below zero leads to revert, one entirely above zero supports keep, and an interval spanning zero requires confirmation. Thus the initial v3 result cannot by itself close the decision. Removing its confirmation while still submitting v3 leaves an unresolved pending decision. More generally, the verifier checks the recorded policy, not whether that policy is scientifically optimal.

C. What the recipient checks

The checks answer progressively stronger questions:

- 1) **Format:** Are required fields present and values well formed?
- 2) **File binding:** Do referenced files exist, and do their bytes match their declared hashes?

- 3) **Decision consistency:** Do parent/candidate references, recalculated statistics, decision rules, and submitted-version identity agree?

Full verification also reports *coverage*: whether every snapshot in the supplied version directory appears in the record. An unrecorded extra snapshot can leave the other checks passing but make coverage partial. Our experiment requires complete coverage for acceptance. This is completeness of the supplied directory, not proof that the producer disclosed every experiment.

Consider changing v2’s recorded mean delta from −318.75 to −317.75, in both the decision log and capsule, then updating the log hash. Format and file binding still pass. Recalculation detects the discrepancy. This particular one-point error would not change the revert verdict; it tests numerical consistency, not the discovery of an incorrectly chosen winner. A separate test changes the submitted code while retaining the old version claim, exercising decision-to-artifact identity.

III. CONTROLLED EVALUATION

A. What is being tested?

We test the checking procedure, not the agent’s ability to play 2048. No new agent rollout is used for this experiment. We copy the same committed synthetic package twelve times and apply one specified change per case (or none for the clean control). Each copy must pass full verification before modification.

The twelve cases comprise nine deliberately introduced faults, two valid controls, and one authenticity-boundary control. Their manifest was committed before first execution, but after inspecting the implementation and development tests. This is an author-designed characterization, not an independent or held-out benchmark. The implementation is unchanged by the study.

We run three checking configurations on each package: **S** checks format only; **B** adds file bindings; **V** performs full verification and requires complete coverage. B uses the production hash functions but omits recalculation, decision rules, cross-field identity and coverage. These are internal ablations of our checker, not measurements of competing systems.

To avoid testing only stale hashes, relevant decision changes are mirrored in the log and capsule and the log hash is refreshed. Swapping the parent/candidate measurements exchanges the referenced paths and hashes while retaining their role labels. Changing the submitted program also refreshes its hashes while retaining the claimed version. All cases, changed-file hashes and actual diagnostics are retained in the companion.

B. Results and interpretation

Table II reports every outcome. S rejects one of nine faults, B rejects three, and V rejects nine. The six additional refusals cover three measurement inconsistencies, an unresolved confirmation, a submitted-version mismatch, and an incomplete snapshot inventory. Thus files can match their declarations while the relationships needed to audit a decision still fail.

TABLE II
CONTROLLED PACKAGES. S: FORMAT; B: FORMAT + FILE HASHES; V: FULL CHECKS INCLUDING COVERAGE. A: ACCEPTS THAT CHECK SCOPE; R: REJECTS.

Package change	S	B	V
None (valid control)	A	A	A
Excluded caches differ; full hashes refreshed	A	A	A
Reward and declaration rewritten together	A	A	A
Unrecognized version status	R	R	R
Referenced measurement file removed	A	R	R
Recorded score direction reversed	A	A	R
Recorded mean delta increased by one	A	A	R
Parent/candidate measurements exchanged	A	A	R
Pending submission left unconfirmed	A	A	R
Submission changed; version claim retained	A	A	R
Recorded snapshot removed	A	R	R
Unrecorded snapshot added	A	A	R

Both valid controls pass: the unchanged package and a package whose excluded bytecode caches differ. This establishes acceptance on these two examples, not a population false-positive rate. Likewise, the fault counts do not estimate detection rates on naturally occurring errors.

Rejection reasons matter. The extra snapshot is rejected for partial coverage, not an integrity error. The missing snapshot is correctly refused, but its diagnostic prefix differs from the manifest’s expected prefix. All nine faults are refused, but only eight match the prespecified diagnostic target. The unchanged expectation and actual output are both preserved; unexpected runtime exceptions abort the study rather than count as detections.

C. A passing rewrite defines the trust boundary

The final control changes an unsigned final reward file and the capsule’s matching reward to 0.5, then refreshes the file hash. V accepts. This is the final benchmark reward, not one of the paired visible-game measurements used for keep-or-revert decisions. The verifier checks agreement with the supplied reward file; it neither authenticates the grader nor recalculates that reward from underlying score details.

This accepted rewrite is important to the interpretation of the nine refusals. Our mechanism checks consistency among supplied declarations and bytes. A producer able to rewrite evidence consistently can still supply false material. Acceptance must therefore not be read as proof that these experiments really happened or that the agent is trustworthy.

IV. SCOPE, RELATED WORK, AND LIMITATIONS

PROV-CONSTRAINTS already defines consistency conditions over provenance histories [2]. Our narrower contribution is a working profile for agent version selection: paired-result recalculation, confirmation state, and the link from a decision to submitted code. We do not claim that adding a provenance record is itself novel.

in-toto verifies cryptographically linked software-supply-chain steps [3]; our unsigned packages provide no comparable

authentication. MLflow records experiment parameters, metrics and artifacts [4]; a tracking system could retain the inputs our checks need. Neither system is an experimental baseline here. Storage, authentication, and decision-consistency checks address different parts of an evidence handoff.

Our results use one synthetic base, author-selected changes and shared checking components. The retained real-run case shows why evidence capture matters, but missing source files prevent its full replay. We have not demonstrated generalization to unseen tasks, reduced human audit effort, independent implementer agreement, or resistance to coordinated falsification. Reproducing an interval also does not certify its statistical validity under adaptive reuse of test seeds.

The Agent-Native Research Artifact companion [5] contains the executable cases, claims, source bindings and exploration history. Historical reward comparisons, development diagnostics, and a separate legacy knowledge-reuse case are preserved there but are not additional trials of this verifier. A downstream agent could use the check outcomes before accepting a supplied version; network-scale governance and authorization remain outside this evaluation.

V. CONCLUSION

A final program and score leave important questions about version selection unanswered. Our record and verifier make several of those questions executable: do the measurements, decision rule and submitted code agree? On the authored cases, full checks reject six faults that format and file-hash checks accept. A consistent unsigned reward rewrite still passes. Together, these outcomes show both the utility and the limit of the approach: auditable decision relationships within a supplied package, not proof of authentic execution.

REFERENCES

- [1] Aiming Lab, *RSI-Exam*, 2026. [Online]. Available: <https://github.com/aiming-lab/RSI-Exam>
- [2] J. Cheney, P. Missier, and L. Moreau, Eds., *Constraints of the PROV Data Model*, W3C Recommendation, Apr. 2013.
- [3] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappel, *in-toto: Providing farm-to-table guarantees for bits and bytes*, in *USENIX Security*, 2019, pp. 1393–1410.
- [4] MLflow, *ML Experiment Tracking*. [Online]. Available: <https://mlflow.org/docs/latest/ml/tracking/>
- [5] ARA Labs, *Agent-Native Research Artifact*. [Online]. Available: <https://github.com/ARA-Labs/Agent-Native-Research-Artifact>